KUBERNETES-06

Task 1 – Create ConfigMap from a directory and inject it into Pod as environment variables

Objective

Create a ConfigMap from a directory containing multiple files and inject all its key-value pairs into a Kubernetes Pod as environment variables.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
  labels:
    app: configmap-demo
spec:
  containers:
    - name: nginx
      image: nginx
      envFrom:
        - configMapRef:
            name: app-config
```

```
root@k8s-master:~# mkdir config-files
root@k8s-master:~# cd config-files
root@k8s-master:~/config-files# echo "nginx" > app.properties
root@k8s-master:~/config-files# echo "production" > environment.properties
root@k8s-master:~/config-files# echo "8080" > port.properties
root@k8s-master:~/config-files# ls
app.properties  environment.properties  port.properties
root@k8s-master:~/config-files# cat app.properties
nginx
root@k8s-master:~/config-files# kubectl create configmap app-config --from-file=.
configmap/app-config created
root@k8s-master:~/config-files# kubectl get configmap
NAME               DATA   AGE
app-config         3      14s
kube-root-ca.crt   1      5d22h
```

```
root@k8s-master:~# kubectl apply -f configmap-pod.yaml
pod/configmap-pod created
root@k8s-master:~# kubectl get pods -o wide
NAME            READY   STATUS    RESTARTS      AGE   IP            NODE          NO
MINATED NODE   READINESS GATES
configmap-pod   1/1     Running   0             38s   10.244.1.84   k8s-worker2   <n
one>           <none>
nginx-pod       1/1     Running   1 (11m ago)   22h   10.244.1.83   k8s-worker2   <n
one>           <none>
root@k8s-master:~# kubectl exec configmap-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-pod
NGINX_VERSION=1.31.8
NJS_VERSION=1.0.2
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
environment.properties=production

port.properties=8080
```

1. **Created a directory containing multiple files.**
2. **Created a ConfigMap using `--from-file`.**
3. **Injected ConfigMap data into a Pod using `envFrom`.**
4. **Verified environment variables using `kubectl exec`.**
5. **Understood how Kubernetes separates configuration from application deployment.**

Command	Purpose
`kubectl get configmap`	List ConfigMaps
`kubectl describe configmap app-config`	Inspect ConfigMap
`kubectl create configmap`	Create ConfigMap
`kubectl apply -f file.yaml`	Create/update resources
`kubectl exec`	Execute commands inside Pod
`printenv`	Display environment variables

Task 2 – Create ConfigMap from a file and mount it as a Volume inside a Pod

Objective

Create a ConfigMap from a configuration file and mount it as a volume inside a Kubernetes Pod. Verify that the configuration data is accessible as a file inside the container.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: config-volume-pod
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config

  volumes:
    - name: config-volume
      configMap:
        name: database-config
```

```
root@k8s-master:~# mkdir config-volume
root@k8s-master:~# cd config-volume
root@k8s-master:~/config-volume# vi database.properties
root@k8s-master:~/config-volume# cat database.properties
DB_HOST=mysql-service
DB_PORT=3306
DB_NAME=company
root@k8s-master:~/config-volume# kubectl create configmap database-config --from-fil
e=database.properties
configmap/database-config created
root@k8s-master:~/config-volume# kubectl get configmap
NAME              DATA   AGE
app-config        3      19m
database-config   1      31s
```

kubectl describe configmap database-config

```
root@k8s-master:~/config-volume# kubectl describe configmap database-config
Name:         database-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
database.properties:
----
DB_HOST=mysql-service
DB_PORT=3306
DB_NAME=company



BinaryData
====

Events:  <none>
```

```
root@k8s-master:~/config-volume# vi config-volume-pod.yaml
root@k8s-master:~/config-volume# kubectl apply -f config-volume-pod.yaml
pod/config-volume-pod created
root@k8s-master:~/config-volume# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS      AGE   IP            NODE          NOM
INATED NODE   READINESS GATES
config-volume-pod   1/1     Running   0             12s   10.244.2.89   k8s-worker1   <no
ne>           <none>
configmap-pod       1/1     Running   0             20m   10.244.1.84   k8s-worker2   <no
ne>           <none>
nginx-pod           1/1     Running   1 (32m ago)   22h   10.244.1.83   k8s-worker2   <no
ne>           <none>
root@k8s-master:~/config-volume# kubectl exec config-volume-pod -- ls /etc/config
database.properties
root@k8s-master:~/config-volume# kubectl exec config-volume-pod -- cat /etc/config/databa
se.properties
DB_HOST=mysql-service
DB_PORT=3306
DB_NAME=company
```

Created a directory named `config-volume`.

1. Created a configuration file named database.properties.
2. Added database configuration details:
    - DB_HOST
    - DB_PORT
    - DB_NAME
3. Created ConfigMap using kubectl create configmap --from-file.
4. Verified ConfigMap using kubectl get configmap and kubectl describe configmap.
5. Created a Pod using YAML configuration.
6. Used volumes to connect the ConfigMap to the Pod.
7. Used volumeMounts to mount configuration data inside /etc/config.
8. Verified the mounted file using kubectl exec.
9. Read the configuration values from inside the container.

Command	Purpose
`kubectl create configmap database-config --from-file=database.properties`	Create ConfigMap
`kubectl get configmap`	List ConfigMaps
`kubectl describe configmap database-config`	View ConfigMap details
`kubectl apply -f config-volume-pod.yaml`	Create Pod
`kubectl get pods`	Check Pod status
`kubectl exec config-volume-pod -- ls /etc/config`	List mounted files

`kubectl exec config-volume-pod -- cat /etc/config/database.properties`	Read mounted configuration

Task 3 – Create Kubernetes Secret and Inject as Environment Variables

Objective

To create a Kubernetes Secret containing sensitive information (username and password) and inject the Secret values into a Pod as environment variables.

```
NAME         TYPE     DATA   AGE
app-secret   Opaque   2      20s
root@k8s-master:~/config-volume# kubectl describe secret app-secret
Name:         app-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  8 bytes
username:  5 bytes
root@k8s-master:~/config-volume# kubectl get secret app-secret -o yaml
apiVersion: v1
data:
  password: YWRtaW4xMjM=
  username: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: "2026-11-12T09:42:05Z"
  name: app-secret
  namespace: default
```

```
root@k8s-master:~/config-volume# kubectl exec secret-env-pod -- printenv USERNAME
Error from server (NotFound): pods "secret-env-pod" not found
root@k8s-master:~/config-volume# kubectl exec secret-env-pod -- printenv PASSWORD
Error from server (NotFound): pods "secret-env-pod" not found
root@k8s-master:~/config-volume# kubectl apply -f secret-env-pod.yaml
error: the path "secret-env-pod.yaml" does not exist
root@k8s-master:~/config-volume# nano secret-env-pod.yaml
root@k8s-master:~/config-volume# kubectl apply -f secret-env-pod.yaml
pod/secret-env-pod created
root@k8s-master:~/config-volume# kubectl get pods
NAME               READY   STATUS    RESTARTS      AGE
config-volume-pod  1/1     Running   0             14m
configmap-pod      1/1     Running   0             35m
nginx-pod          1/1     Running   1 (46m ago)   23h
secret-env-pod     1/1     Running   0             9s
root@k8s-master:~/config-volume# kubectl exec secret-env-pod -- printenv USERNAME
admin
root@k8s-master:~/config-volume# kubectl exec secret-env-pod -- printenv PASSWORD
admin123
root@k8s-master:~/config-volume#
```

1. Created a Kubernetes Secret named `app-secret`.
2. Used `kubectl create secret generic` to create the Secret.
3. Stored sensitive information such as username and password.
4. Used `--from-literal` to provide Secret key-value pairs.
5. Verified Secret using `kubectl get secrets`.
6. Inspected Secret details using `kubectl describe secret`.
7. Understood Base64 encoding of Kubernetes Secret data.
8. Decoded Secret values using the `base64 -d` command.
9. Created a Pod YAML file named `secret-env-pod.yaml`.
10. Used `secretKeyRef` to inject Secret values into environment variables.
11. Created the Pod using `kubectl apply -f`.
12. Learned how applications access sensitive credentials through environment variables.

Task 4 – Create Kubernetes Secret using YAML and Mount it as a Volume inside a Pod.

. Objective

Create a Kubernetes Secret using a YAML file, mount it as a volume inside a Pod, and verify that username and password are accessible as files inside the container.

```
vi secret.yaml
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: database-secret
type: Opaque
stringData:
  username: admin
  password: admin123
```

```
vi secret-volume-pod.yaml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secret
          readOnly: true

  volumes:
    - name: secret-volume
      secret:
        secretName: database-secret
```

```
secret/database-secret created
root@k8s-master:~# kubectl get secrets
NAME              TYPE     DATA   AGE
app-secret        Opaque   2      15m
database-secret   Opaque   2      8s
root@k8s-master:~# kubectl describe secret database-secret
Name:         database-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  8 bytes
username:  5 bytes
root@k8s-master:~# vi secret-volume-pod.yaml
root@k8s-master:~# kubectl apply -f secret-volume-pod.yaml
pod/secret-volume-pod created
```

```
password:  8 bytes
username:  5 bytes
root@k8s-master:~# vi secret-volume-pod.yaml
root@k8s-master:~# kubectl apply -f secret-volume-pod.yaml
pod/secret-volume-pod created
root@k8s-master:~# kubectl get pods
NAME                READY   STATUS    RESTARTS      AGE
config-volume-pod   1/1     Running   0             25m
configmap-pod       1/1     Running   0             46m
nginx-pod           1/1     Running   1 (57m ago)   23h
secret-env-pod      1/1     Running   0             10m
secret-volume-pod   1/1     Running   0             9s
root@k8s-master:~# kubectl exec secret-volume-pod -- ls /etc/secret
password
username
```

1. Created a YAML file named secret.yaml.
2. Defined a Kubernetes Secret using apiVersion: v1 and kind: Secret.
3. Used type: Opaque to create a generic Secret.
4. Used stringData to provide username and password in plain text in the YAML manifest.
5. Created the Secret using kubectl apply -f secret.yaml.
6. Verified Secret creation using kubectl get secrets.
7. Inspected Secret details using kubectl describe secret.

8. Created a Pod YAML file named secret-volume-pod.yaml.
9. Used volumes to connect the Secret to the Pod.
10. Used volumeMounts to mount Secret data inside /etc/secret.
11. Used readOnly: true to prevent modification of mounted Secret files.

Task 5 – Inject ConfigMap as Environment Variables and Secret as Files in the Same Pod

. Objective

To inject ConfigMap data into a Kubernetes Pod as environment variables and mount Secret data as files inside the same Pod, ensuring both configuration and sensitive information are accessible to the application

```
root@k8s-master:~# vi combined-pod.yaml
root@k8s-master:~# kubectl apply -f combined-pod.yaml
pod/combined-pod created
root@k8s-master:~# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS      AGE   IP            NODE          NOM
INATED NODE   READINESS GATES
combined-pod        1/1     Running   0             9s    10.244.2.91   k8s-worker1   <no
ne>           <none>
config-volume-pod   1/1     Running   0             36m   10.244.2.89   k8s-worker1   <no
ne>           <none>
configmap-pod       1/1     Running   0             57m   10.244.1.84   k8s-worker2   <no
ne>           <none>
nginx-pod           1/1     Running   1 (68m ago)   23h   10.244.1.83   k8s-worker2   <no
ne>           <none>
secret-env-pod      1/1     Running   0             22m   10.244.1.85   k8s-worker2   <no
ne>           <none>
secret-volume-pod   1/1     Running   0             11m   10.244.2.90   k8s-worker1   <no
ne>           <none>
root@k8s-master:~# kubectl exec combined-pod -- printenv APP_CONFIG
nginx

root@k8s-master:~# kubectl exec combined-pod -- printenv ENVIRONMENT_CONFIG
production

root@k8s-master:~# kubectl exec combined-pod -- printenv PORT_CONFIG
8080
```

```
root@k8s-master:~# kubectl exec combined-pod -- printenv PORT_CONFIG
8080

root@k8s-master:~# kubectl exec combined-pod -- ls /etc/secret
password
username
root@k8s-master:~# kubectl exec combined-pod -- cat /etc/secret/username
adminroot@k8s-master:~#
```

1. Verified existing ConfigMap and Secret resources.
2. Created a combined Pod YAML file named combined-pod.yaml.
3. Used configMapKeyRef to inject individual ConfigMap values as environment variables.
4. Used volumeMounts to mount Secret data inside the container.
5. Used volumes to connect the Secret to the Pod.
6. Used readOnly: true to prevent modification of mounted Secret files.
7. Created the Pod using kubectl apply -f.
8. Verified ConfigMap environment variables using kubectl exec.
9. Verified Secret files using ls and cat.